STAT2003: Assignment 1

Samantha Clements and Kafula Chewe

2026-04-03

«««< HEAD

=======

Machine Learning Algorithms Comparison

A randomised block design (RBD) experiment was conducted to compare four machine learning algorithms for predicting credit card default using the Default dataset from the ISLR package (James et al. 2013). The experimental factor is algorithm type, with four levels: Logistic Regression (LR), Random Forest (RF), Naive Bayes (NB), and Support Vector Machine (SVM). The experimental design is illustrated in the DOE Diagram (Figure 1.1).

The primary response is AUC-ROC, referred to as ROC in caret's output. The Default dataset reveals a severe class-imbalance leading to misleading overall accuracy. Models predicting non-defaulters for all observations achieve roughly 97% accuracy yet have no discriminatory value. AUC-ROC measures each model's ability to distinguish defaulters from non-defaulters and is robust to class imbalance, making it the most appropriate metric (James et al. 2013).

The experiment is subject to three principles of experimental design (Montgomery 2020). Randomisation was conducted through random observation assignment to cross-validation (CV) folds with set.seed(123). Replication is achieved via repeated 10-fold CV with 5 repeats, producing 50 resamples. Blocking treats each resample as a block; all models are evaluated on identical splits, removing fold-to-fold variation. Residual diagnostics are conducted prior to formal RBD ANOVA interpretation.

(198 words)

QUESTION 2

The results below show differences in the ROC averages across all four algorithms. Logistic Regression and Naive Bayes had the highest ROC values and showed fairly stable performance across all blocks. Random Forrest performed moderately whereas SVM showed inconsistent behaviour across resamples. SVM may appear to be performing well in some cases, however, this may be misleading due to the class imbalance that exists in the dataset, for instance, a model that predicts most observations as "No default" can still achieve a reasonable accuracy or ROC despite being unable to accurately predict the minority class.

The results show that the algorithms do not perform equally with Logistic Regression and Naive Bayes demonstrating a more stable and stronger performance in this experiment. (308 words)

```
##   default student   balance    income
## 1      No      No  729.5265 44361.625
## 2      No     Yes  817.1804 12106.135
## 3      No      No 1073.5492 31767.139
## 4      No      No  529.2506 35704.494
## 5      No      No  785.6559 38463.496
## 6      No     Yes  919.5885  7491.559
```

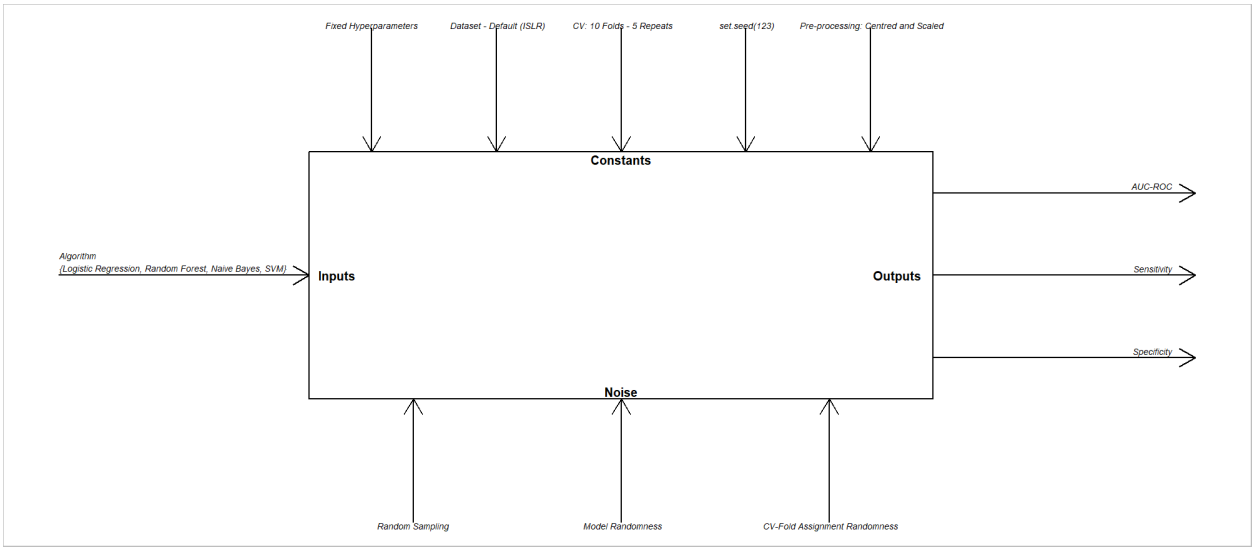


Figure 1: Figure 1.1: DOE diagram for Default Experiment

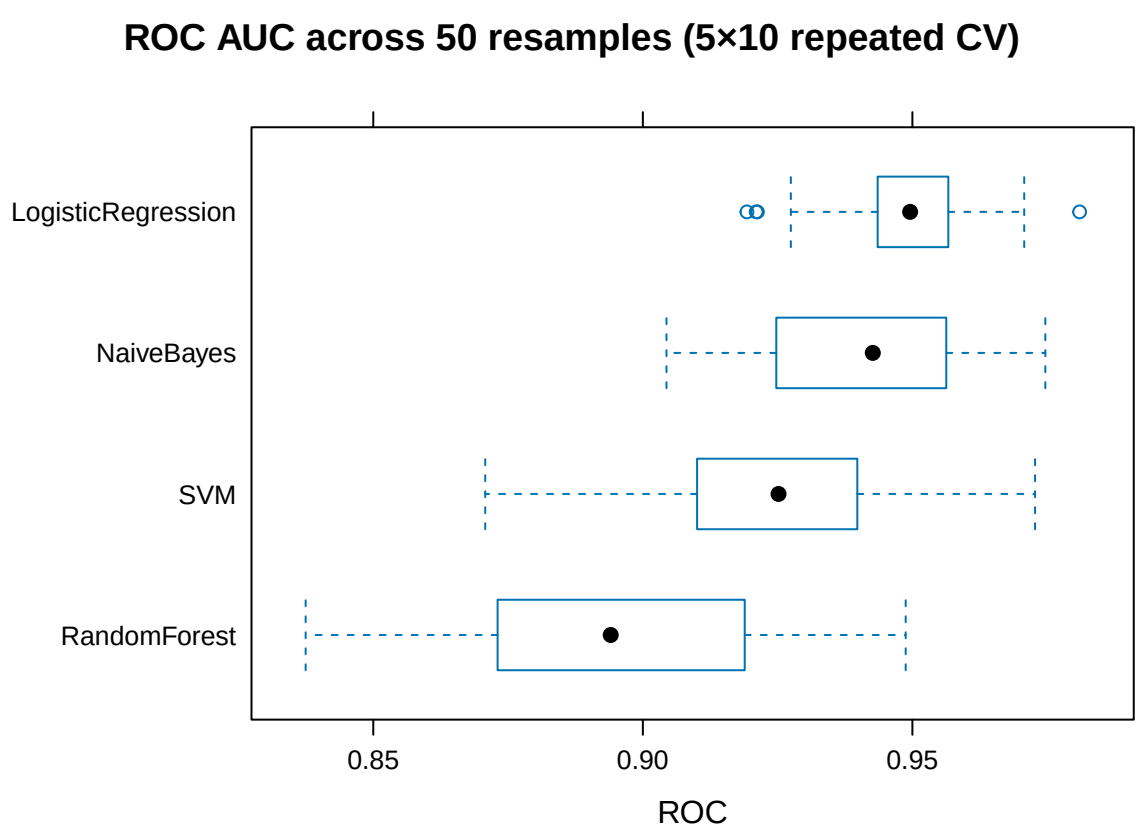
```
##
##   No  Yes
## 9667  333
```

```r
set.seed(123)
ctrl <- trainControl(method = "repeatedcv",
                     number = 10,
                     repeats = 5,
                     summaryFunction = twoClassSummary,
                     classProbs = TRUE,
                     savePredictions = TRUE)
```

```
## note: only 2 unique complexity parameters in default grid. Truncating the grid to 2 .
```

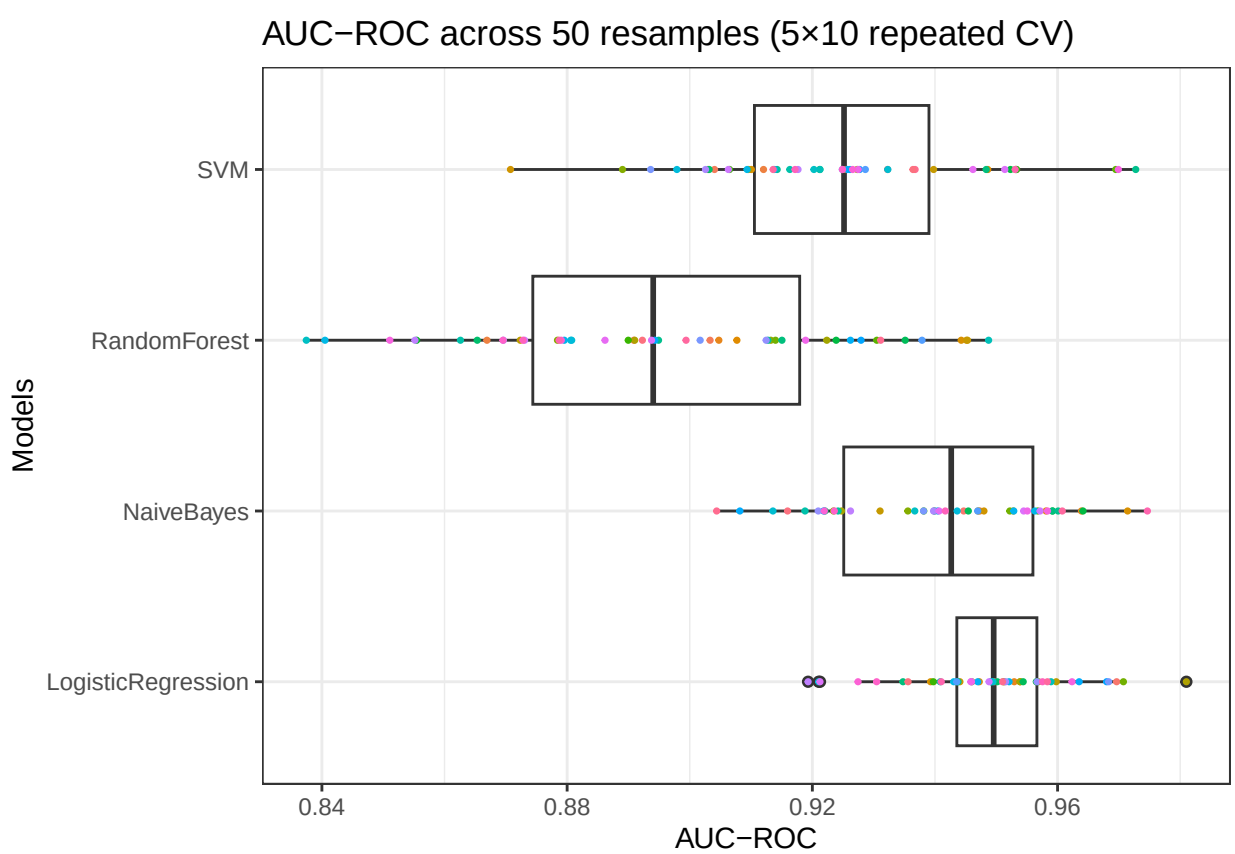
```
##
## Call:
## summary.resamples(object = res)
##
## Models: LogisticRegression, RandomForest, NaiveBayes, SVM
## Number of resamples: 50
##
## ROC
##                        Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## LogisticRegression 0.9193068 0.9435696 0.9495785 0.9491588 0.9566296 0.9810097
## RandomForest       0.8374589 0.8744026 0.8940430 0.8964833 0.9179341 0.9487499
## NaiveBayes         0.9043903 0.9251208 0.9426542 0.9416489 0.9559826 0.9746534
## SVM                0.8707656 0.9105552 0.9251687 0.9257537 0.9390234 0.9727500
##                    NA's
## LogisticRegression    0
## RandomForest          0
## NaiveBayes            0
## SVM                   0
##
```

```
## Sens
##                          Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## LogisticRegression 0.9917184 0.9948240 0.9958635 0.9957379 0.9968976 1.0000000
## RandomForest       0.9793175 0.9875905 0.9896587 0.9896146 0.9917270 0.9958635
## NaiveBayes         0.9927536 0.9968976 0.9979317 0.9978277 0.9989648 1.0000000
## SVM                1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000
##                    NA's
## LogisticRegression    0
## RandomForest          0
## NaiveBayes            0
## SVM                   0
##
## Spec
##                           Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## LogisticRegression 0.20588235 0.2727273 0.3030303 0.3195900 0.3609626 0.4848485
## RandomForest       0.20588235 0.2727273 0.3235294 0.3216043 0.3636364 0.4705882
## NaiveBayes         0.06060606 0.1212121 0.1791444 0.1686453 0.2058824 0.3030303
## SVM                0.00000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
##                    NA's
## LogisticRegression    0
## RandomForest          0
## NaiveBayes            0
## SVM                   0
```

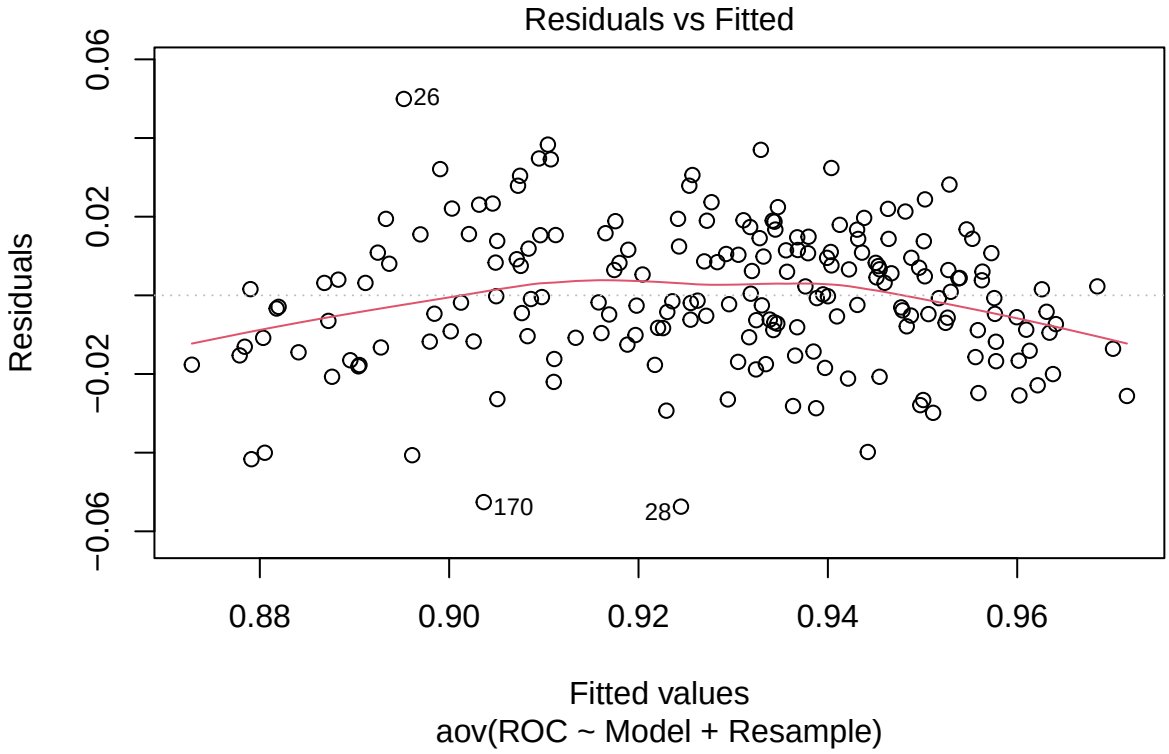


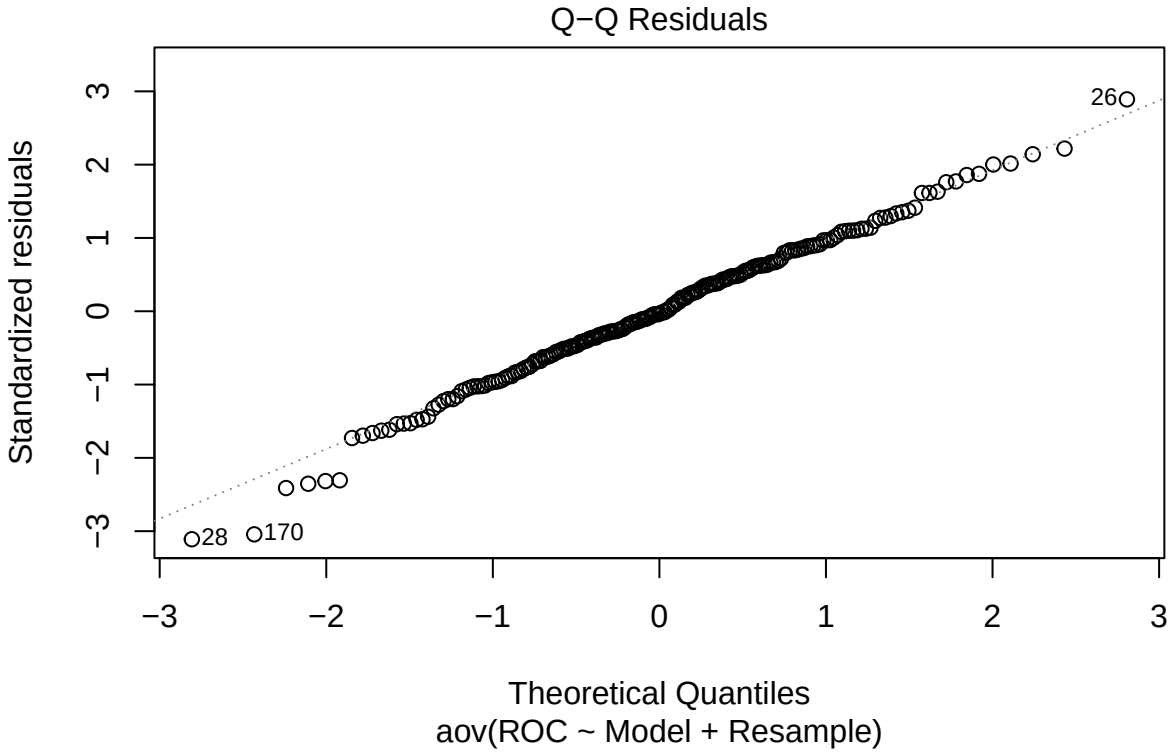
```r
res_df <- as.data.frame(res)
# head(res_df)

res_long <- pivot_longer(res_df,
                         cols = -Resample,
                         names_to = "Model", values_to = "ROC")
```

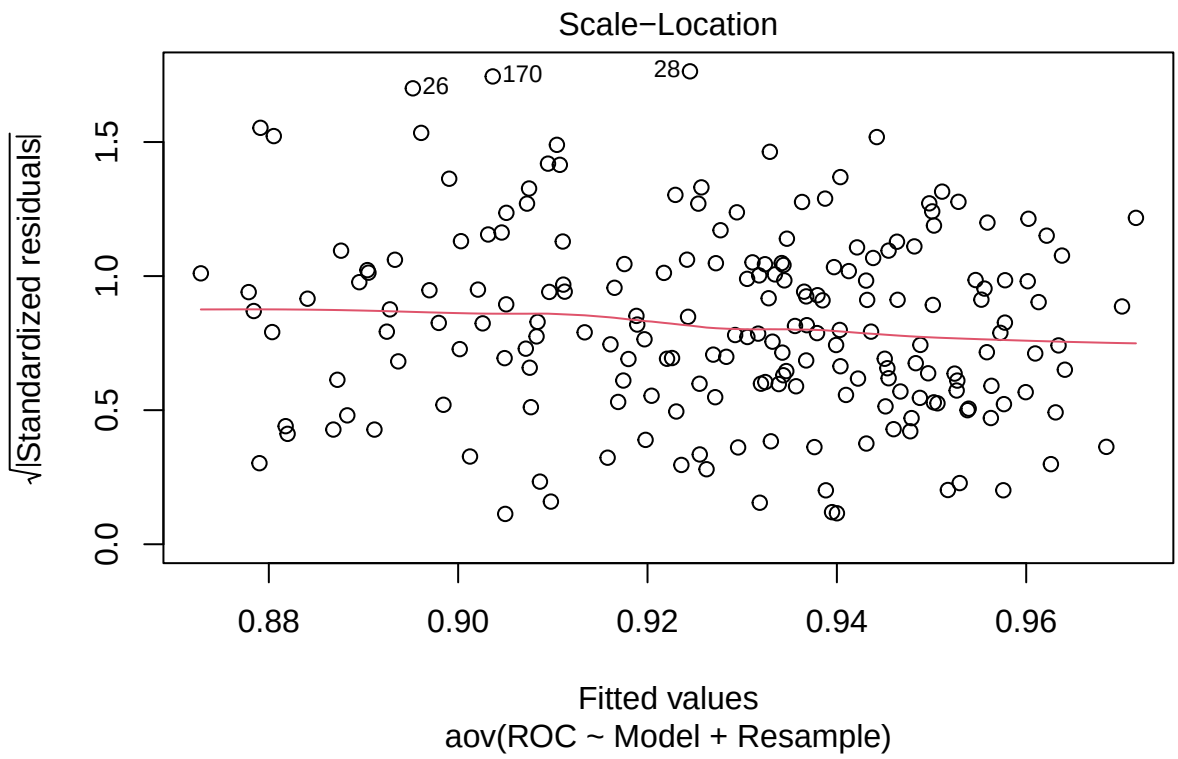
```r
res_long |>
  ggplot() +
  aes(x = ROC, y = Model) +
  geom_boxplot() +
  geom_point(aes(colour = Resample), size = 0.6, show.legend = FALSE) +
  labs(y     = "Models",
       x     = "AUC-ROC",
       title = "AUC-ROC across 50 resamples (5×10 repeated CV)") +
  theme_bw()
```

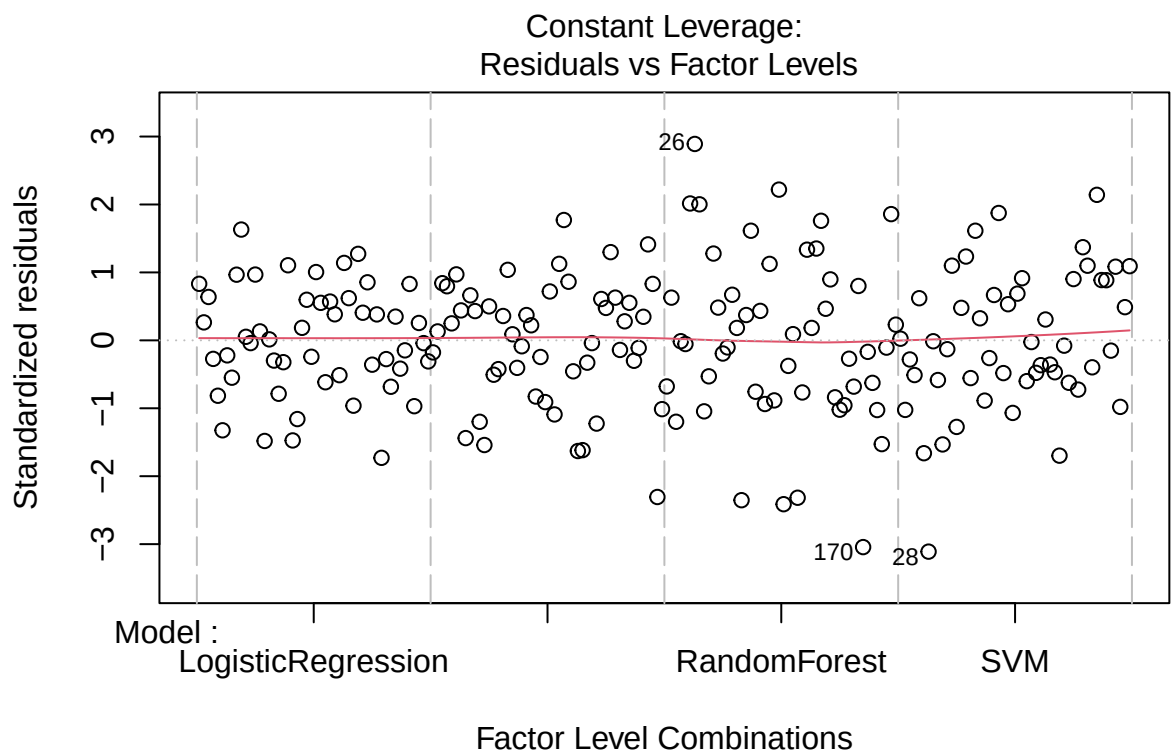



```r
res_aov <- aov(ROC ~ Model + Resample, data = res_long)

# Visual checks
plot(res_aov)
```


Residuals vs Fitted
Residuals
0.06
0.02
−0.02
−0.06
26
170
28
0.88
0.90
0.92
0.94
0.96
Fitted values
aov(ROC ~ Model + Resample)


Q−Q Residuals
Standardized residuals
Theoretical Quantiles
aov(ROC ~ Model + Resample)
26
28
170
−3
−2
−1
0
1
2
3


Scale−Location
26
170
28
√|Standardized residuals|
0.0
0.5
1.0
1.5
0.88
0.90
0.92
0.94
0.96
Fitted values
aov(ROC ~ Model + Resample)




```r
#head(res_aov)

# Formal normality test on residuals
shapiro.test(residuals(res_aov))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(res_aov)
## W = 0.99398, p-value = 0.5984
```

```r
# Formal homogeneity of variance test
bartlett.test(ROC ~ Model, data = res_aov$model)
```

```
##
##  Bartlett test of homogeneity of variances
##
## data:  ROC by Model
## Bartlett's K-squared = 34.734, df = 3, p-value = 1.387e-07
```

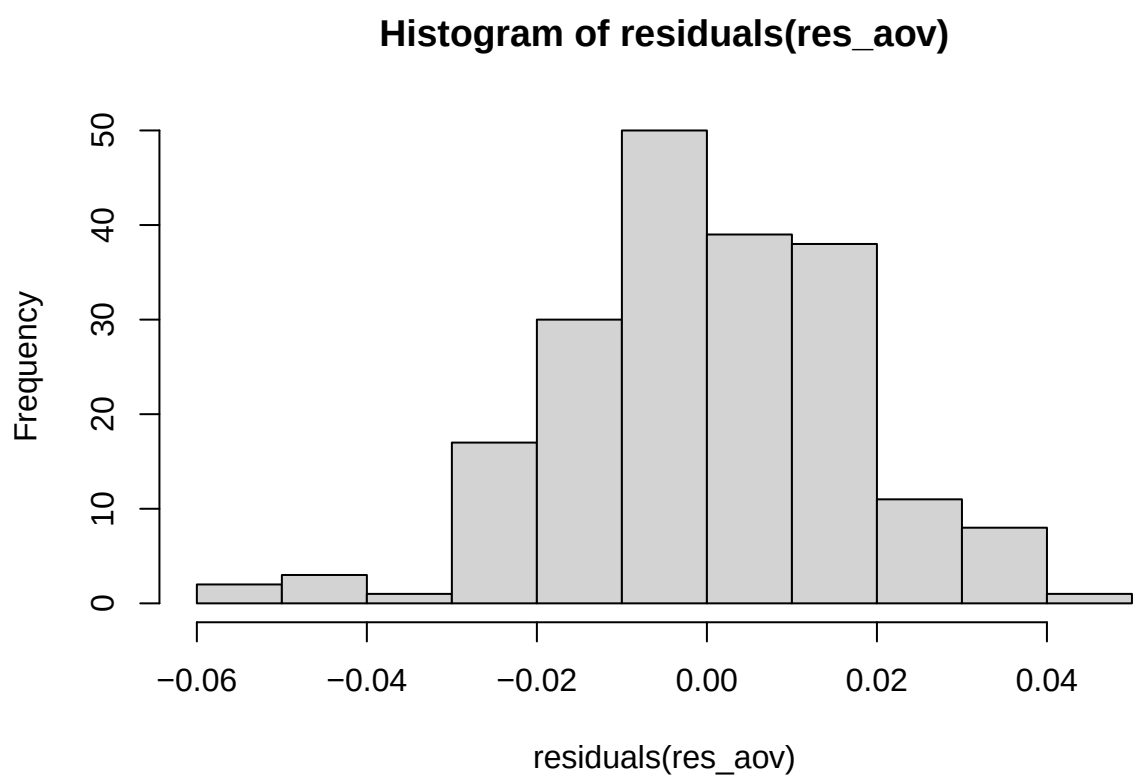
```r
leveneTest(ROC ~ Model, data = res_long)
```

```
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
```

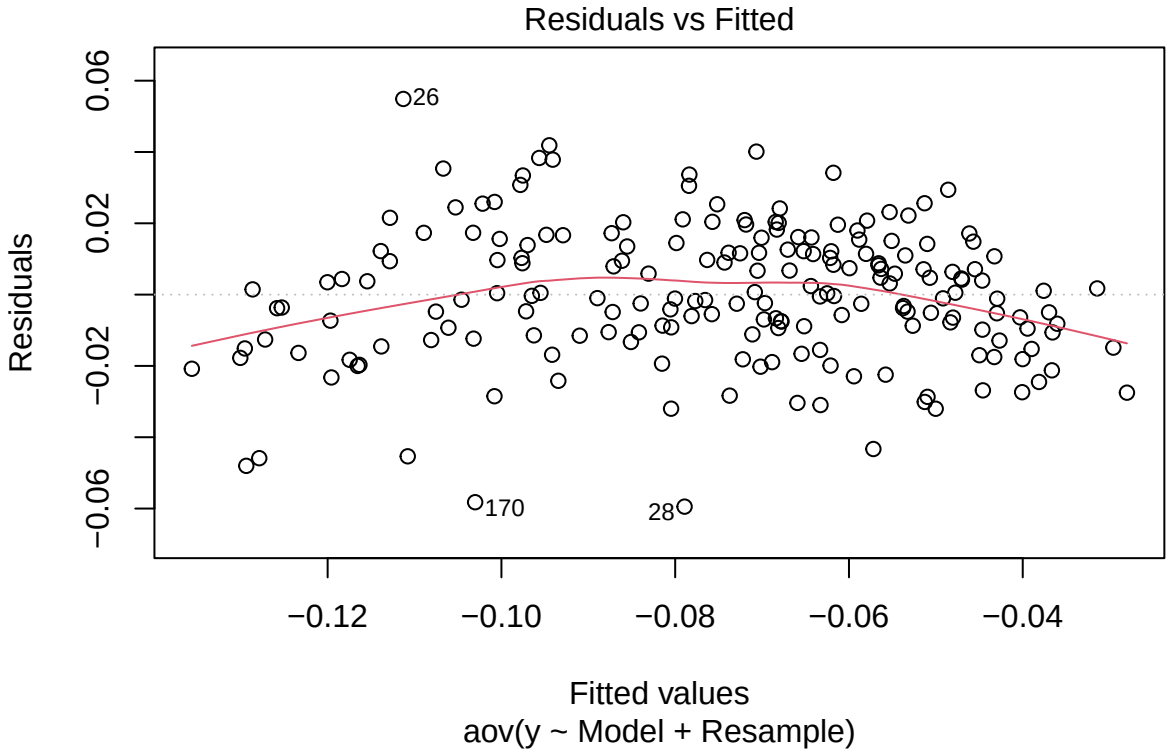
```
## Levene's Test for Homogeneity of Variance (center = median)
##        Df F value    Pr(>F)
## group   3  12.538 1.558e-07 ***
##       196
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

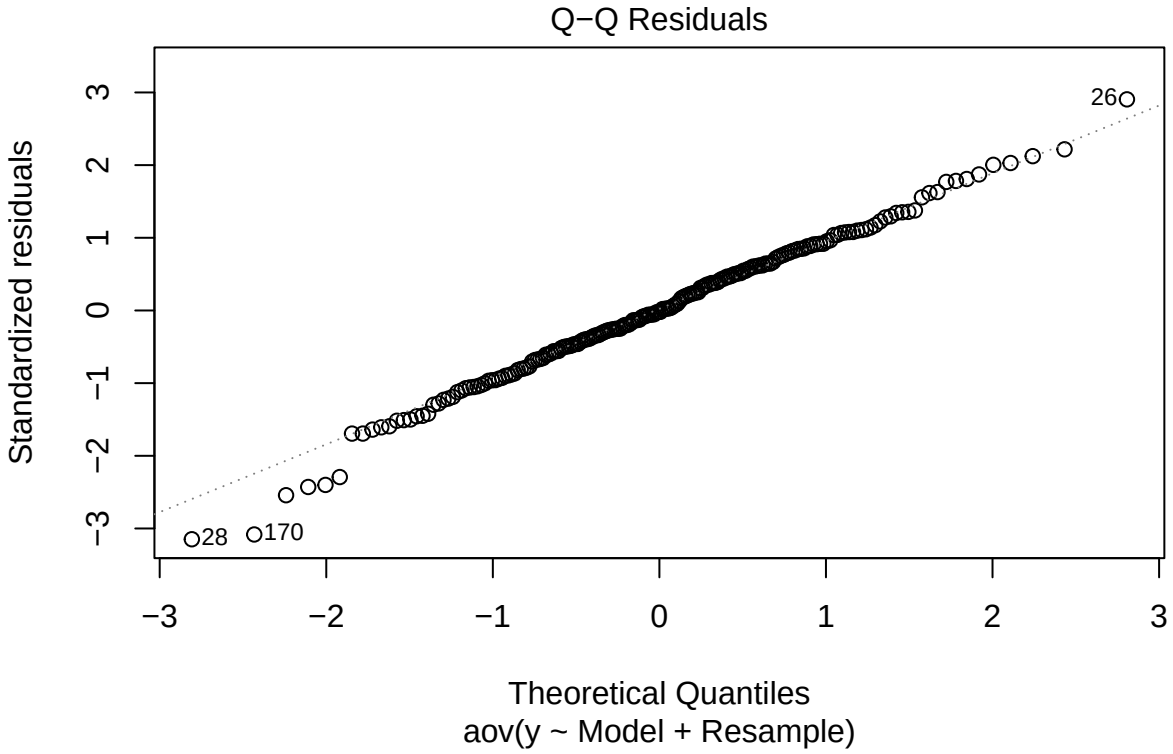
Shapiro: $p > 0.05$, do not reject H0, normality is satisfied Bartlett: $p < 0.05$, reject H0, equal variance is not satisfied

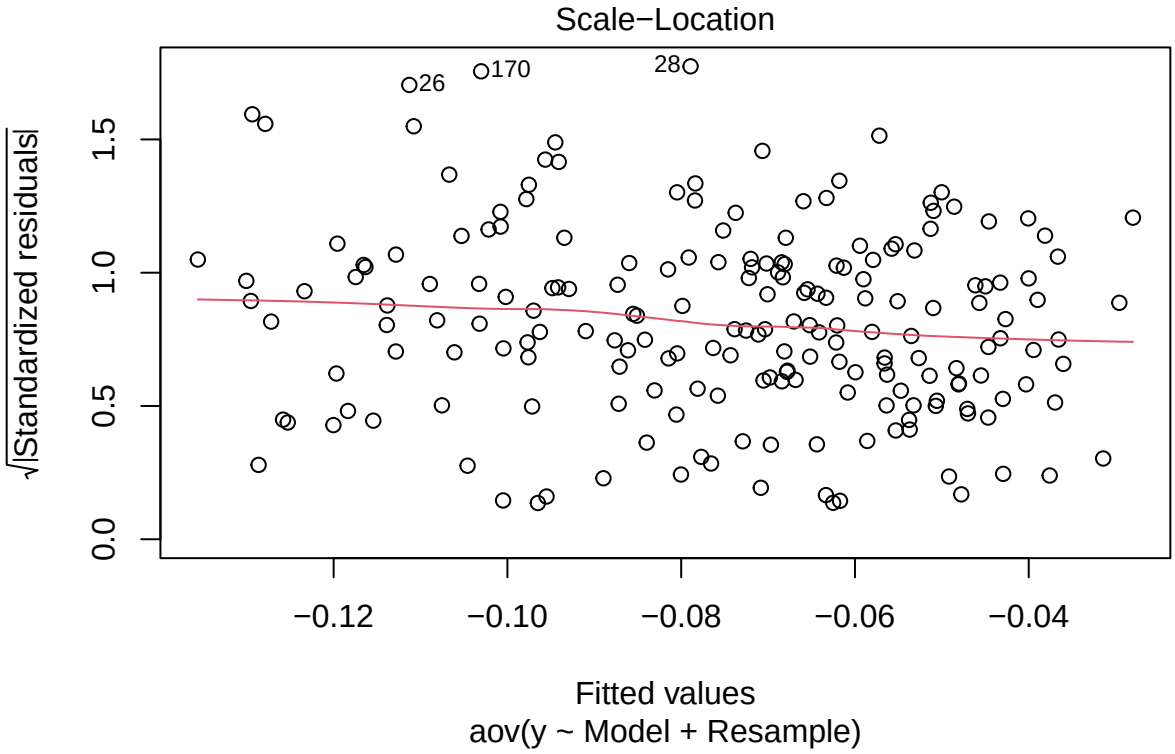
```r
hist(residuals(res_aov))
```

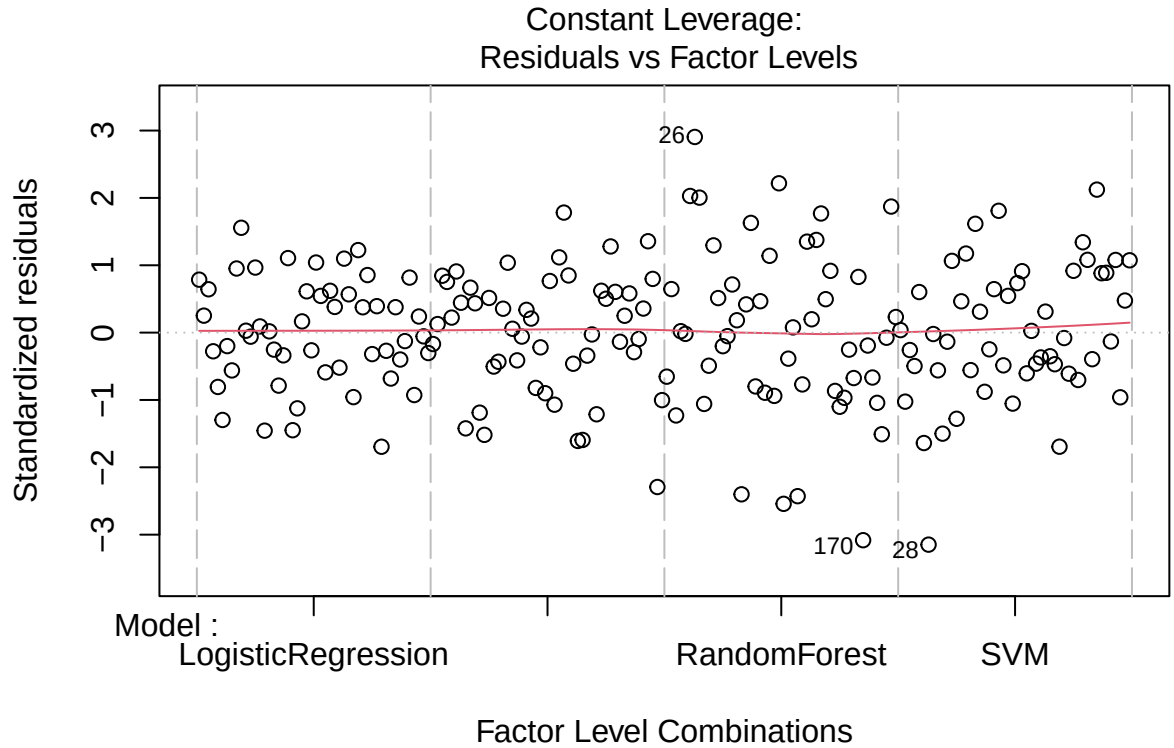



```r
# log transform, H0 = equal medians / equal geometric means

y <- log(res_long$ROC)

res_log_aov <- aov(y ~ Model + Resample, data = res_long)


plot(res_log_aov)
```


Residuals vs Fitted
Residuals
0.06
0.02
−0.02
−0.06
26
170
28
−0.12
−0.10
−0.08
−0.06
−0.04
Fitted values
aov(y ~ Model + Resample)


Q−Q Residuals
Standardized residuals
Theoretical Quantiles
aov(y ~ Model + Resample)
26
28
170
−3
−2
−1
0
1
2
3


Scale−Location
26
170
28
√|Standardized residuals|
0.0
0.5
1.0
1.5
−0.12
−0.10
−0.08
−0.06
−0.04
Fitted values
aov(y ~ Model + Resample)




```r
shapiro.test(residuals(res_log_aov))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(res_log_aov)
## W = 0.99256, p-value = 0.4054
```

```r
bartlett.test(y ~ Model, data = res_log_aov$model)
```

```
##
##  Bartlett test of homogeneity of variances
##
## data:  y by Model
## Bartlett's K-squared = 39.728, df = 3, p-value = 1.217e-08
```

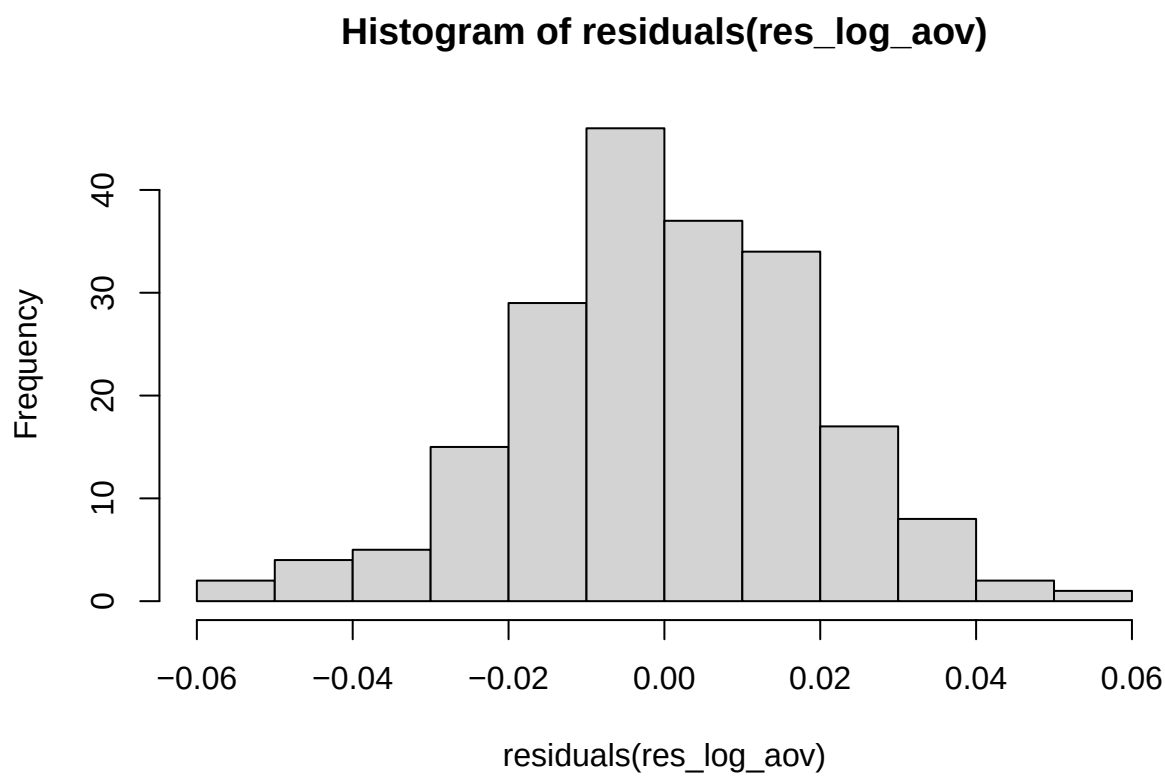
```r
leveneTest(y ~ Model, data = res_long)
```

```
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##        Df F value    Pr(>F)
```

```
## group   3  14.181 2.111e-08 ***
##       196
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
hist(residuals(res_log_aov))
```




```r
friedman.test(ROC ~ Model | Resample, data = res_long)
```

```
##
##  Friedman rank sum test
##
## data:  ROC and Model and Resample
## Friedman chi-squared = 74.112, df = 3, p-value = 5.616e-16
```

QUESTION 3 Based on the experimental results, Logistic Regression is recommended for predicting credit card default. It had the highest ROC and showed consistency across resamples (blocks), demonstrating strong discrimination and stability. Though Naive Bayes performed well, Logistic Regression had the best overall performance. A limitation of this recommendation is that it is based on a singular dataset and the chosen resampling design, therefore, the results may differ when presented with new data. (71 words)

References

Kleijnen, Jack P. C. 2008. *Design and Analysis of Simulation Experiments*. 1st ed. New York: Springer-Verlag.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2013. *An Introduction to Statistical Learning*. Vol. 112. New York: Springer.

Montgomery, Douglas C. 2020. *Design and Analysis of Experiments*. 1st ed. Hoboken, NJ: Wiley.

Extra Stuff

Previous Q2p1

To conduct this experiment a Randomised Block Design (RBD) was used to compare the four different machine learning algorithms on their ability to predict credit card default. The experimental factor was the algorithm with the four levels being represented by the the machine learning algorithms: Logistic Regression, Random Forrest, Naive Bayes and Support Vector Machine. The response variable is ROC AUC which explains how good the algorithms are determining whether someone defaulted or not, a higher value indicates good performance. ROC AUC was chosen because the data shows class imbalance, that is, fewer default cases than non-default cases (more No than Yes), hence, making accuracy an unreliable performance metric. To control background variation due to random partitioning, that is, one split making one algorithm look better than another the experiment contains repeated 10-fold cross validation with 3 repeats, therefore, creating 30 resampling blocks. To ensure that it was a fair comparison, within each block all four algorithms were tested under the same split. This means the experiment follows the principles of randomisation by resampling, replication through repeated evaluation and blocking as the shared splits act as blocks.